Customer Shopping Behaviour Analysis

1. Project Overview

This project analyzes customer shopping behavior using transactional data from 3,900 purchases across various product categories. The goal is to uncover insights into spending patterns, customer segments, product preferences, and subscription behaviour, guiding strategic business decisions.

2. Dataset Summary

- Rows: 3,900 - Columns: 18 - Key Features:
- Customer demographics (Age, Gender, Location, Subscription Status)
- Purchase details (Item Purchased, Category, Purchase Amount, Season, Size, Color)
- Shopping behavior (Discount Applied, Promo Code Used, Previous Purchases, Frequency of Purchases, Review Rating, Shipping Type)
- Missing Data: 37 values in Review Rating column

3. Exploratory Data Analysis using Python

We began with data preparation and cleaning in Python:

- **Data Loading:** Imported the dataset using `pandas`.

- **Initial Exploration:** Used `df.info()` to check structure and `.describe()` for summary statistics.

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Shipping Type	Discou Appli
count	3900.000000	3900.000000	3900	3900	3900	3900.000000	3900	3900	3900	3900	3863.000000	3900	3900	39
unique	NaN	NaN	2	25	4	NaN	50	4	25	4	NaN	2	6	
top	NaN	NaN	Male	Blouse	Clothing	NaN	Montana	M	Olive	Spring	NaN	No	Free Shipping	
freq	NaN	NaN	2652	171	1737	NaN	96	1755	177	999	NaN	2847	675	22
mean	1950.500000	44.068462	NaN	NaN	NaN	59.764359	NaN	NaN	NaN	NaN	3.750065	NaN	NaN	N
std	1125.977353	15.207589	NaN	NaN	NaN	23.685392	NaN	NaN	NaN	NaN	0.716983	NaN	NaN	N
min	1.000000	18.000000	NaN	NaN	NaN	20.000000	NaN	NaN	NaN	NaN	2.500000	NaN	NaN	N
25%	975.750000	31.000000	NaN	NaN	NaN	39.000000	NaN	NaN	NaN	NaN	3.100000	NaN	NaN	N
50%	1950.500000	44.000000	NaN	NaN	NaN	60.000000	NaN	NaN	NaN	NaN	3.800000	NaN	NaN	N
75%	2925.250000	57.000000	NaN	NaN	NaN	81.000000	NaN	NaN	NaN	NaN	4.400000	NaN	NaN	N
max	3900.000000	70.000000	NaN	NaN	NaN	100.000000	NaN	NaN	NaN	NaN	5.000000	NaN	NaN	N

Discount Applied	Promo Code Used	Previous Purchases	Payment Method	Frequency of Purchases
3900	3900	3900.000000	3900	3900
2	2	NaN	6	7
No	No	NaN	PayPal	Every 3 Months
2223	2223	NaN	677	584
NaN	NaN	25.351538	NaN	NaN
NaN	NaN	14.447125	NaN	NaN
NaN	NaN	1.000000	NaN	NaN
NaN	NaN	13.000000	NaN	NaN
NaN	NaN	25.000000	NaN	NaN
NaN	NaN	38.000000	NaN	NaN
NaN	NaN	50.000000	NaN	NaN

- **Missing Data Handling:** Checked for null values and imputed missing values in the `Review Rating` column using the median rating of each product category.

- **Column Standardization:** Renamed columns to **snake case** for better readability and documentation.

- **Feature Engineering:**

    - ○ Created **age_group** column by binning customer ages.

    - ○ Created **purchase_frequency_days** column from purchase data.

- **Data Consistency Check:** Verified if `discount_applied` and `promo_code_used` were redundant; dropped `promo_code_used`.

- **Database Integration:** Connected Python script to MYSQL and loaded the cleaned DataFrame into the database for SQL analysis.

4. Data Analysis using SQL (Business Transactions)

We performed structured analysis in MYSQL to answer key business questions:

1. **Revenue by Gender** – Compared total revenue generated by male vs. female customers.

```sql
-- Q1. What is the total revenue generated by male vs. female customers?
select gender, SUM(purchase_amount) as revenue
from customer_staging1
group by gender;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

gender	revenue
Male	157890
Female	75191

2. **High-Spending Discount Users** – Identified customers who used discounts but still spent above the average purchase amount.

```sql
-- Q2. Which customers used a discount but still spent more than the average purchase amount?
SELECT customer_id, purchase_amount
FROM customer_staging1
WHERE discount_applied = 'Yes'
AND purchase_amount >= (SELECT AVG(purchase_amount) from customer_staging1);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

customer_id	purchase_amount
2	64
3	73
4	90
7	85
9	97
12	68
13	72
16	81

3. **Top 5 Products by Rating** – Found products with the highest average review ratings.

```sql
-- Q3. Which are the top 5 products with the highest average review rating?
SELECT item_purchased, ROUND(AVG(review_rating),2) AS "Average Product Rating"
FROM customer_staging1
GROUP BY item_purchased
ORDER BY AVG(review_rating) DESC
LIMIT 5;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

item_purchased	Average Product Rating
Gloves	3.86
Sandals	3.84
Boots	3.82
Hat	3.8
Skirt	3.78

4. **Shipping Type Comparison** – Compared average purchase amounts between Standard and Express shipping.

```sql
-- Q4. Compare the average Purchase Amounts between Standard and Express Shipping.
SELECT shipping_type,
ROUND(AVG(purchase_amount),2)
FROM customer_staging1
WHERE shipping_type IN ('Standard','Express')
GROUP BY shipping_type;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

shipping_type	ROUND(AVG(purchase_amount),2)
Express	60.48
Standard	58.46

5. **Subscribers vs. Non-Subscribers** – Compared average spend and total revenue across subscription status.

```sql
-- Q5. Do subscribed customers spend more? Compare average spend and total revenue
-- between subscribers and non-subscribers.
SELECT subscription_status,
       COUNT(customer_id) AS total_customers,
       ROUND(AVG(purchase_amount),2) AS avg_spend,
       ROUND(SUM(purchase_amount),2) AS total_revenue
FROM customer_staging1
GROUP BY subscription_status
ORDER BY total_revenue,avg_spend DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

subscription_status	total_customers	avg_spend	total_revenue
Yes	1053	59.49	62645
No	2847	59.87	170436

6. **Discount-Dependent Products** – Identified 5 products with the highest percentage of discounted purchases.

```sql
-- Q6. Which 5 products have the highest percentage of purchases with discounts applied?
SELECT item_purchased,
       ROUND(100.0 * SUM(CASE WHEN discount_applied = 'Yes' THEN 1 ELSE 0 END)/COUNT(*),2) AS discount_rate
FROM customer_staging1
GROUP BY item_purchased
ORDER BY discount_rate DESC
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

item_purchased	discount_rate
Hat	50.00
Sneakers	49.66
Coat	49.07
Sweater	48.17
Pants	47.37

7. **Customer Segmentation** – Classified customers into New, Returning, and Loyal segments based on purchase history.

```
-- Q7. Segment customers into New, Returning, and Loyal based on their total
-- number of previous purchases, and show the count of each segment.
WITH customer_type AS (
SELECT customer_id, previous_purchases,
CASE
    WHEN previous_purchases = 1 THEN 'New'
    WHEN previous_purchases BETWEEN 2 AND 10 THEN 'Returning'
    ELSE 'Loyal'
    END AS customer_segment
FROM customer_staging1)

select customer_segment,count(*) AS "Number of Customers"
from customer_type
group by customer_segment;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_segment	Number of Customers
Loyal	3116
Returning	701
New	83

8. **Top 3 Products per Category –** Listed the most purchased products within each category.

```sql
-- Q8. What are the top 3 most purchased products within each category?
WITH item_counts AS (
    SELECT category,
           item_purchased,
           COUNT(customer_id) AS total_orders,
           ROW_NUMBER() OVER (PARTITION BY category ORDER BY COUNT(customer_id) DESC) AS item_rank
    FROM customer_staging1
    GROUP BY category, item_purchased
)
SELECT item_rank,category, item_purchased, total_orders
FROM item_counts
WHERE item_rank <=3;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

item_rank	category	item_purchased	total_orders
1	Accessories	Jewelry	171
2	Accessories	Sunglasses	161
3	Accessories	Belt	161
1	Clothing	Blouse	171
2	Clothing	Pants	171
3	Clothing	Shirt	169
1	Footwear	Sandals	160

9. **Repeat Buyers & Subscriptions –** Checked whether customers with >5 purchases are more likely to subscribe.

```sql
-- Q9. Are customers who are repeat buyers (more than 5 previous purchases) also likely to subscribe?
SELECT subscription_status,
       COUNT(customer_id) AS repeat_buyers
FROM customer_staging1
WHERE previous_purchases > 5
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

subscription_status	repeat_buyers
Yes	958
No	2518

10. **Revenue by Age Group** – Calculated total revenue contribution of each age group.

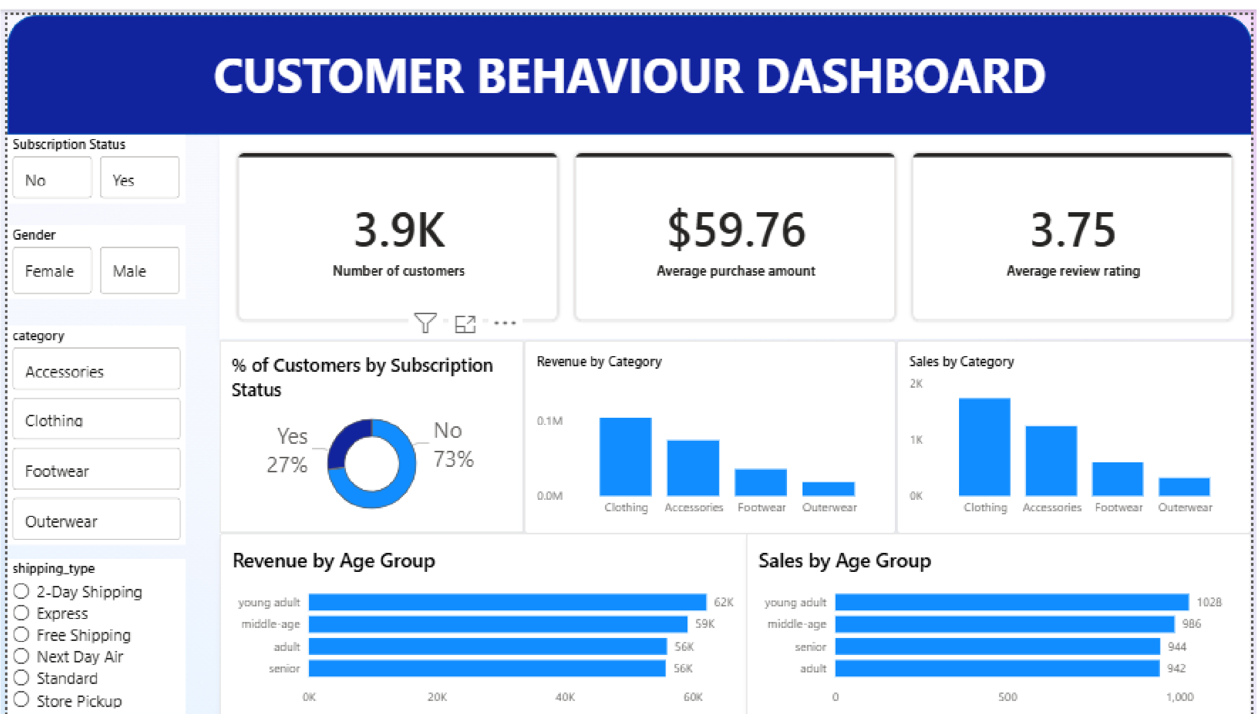
```sql
-- Q10. What is the revenue contribution of each age group?
SELECT
    age_group,
    SUM(purchase_amount) AS total_revenue
FROM customer_staging1
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

age_group	total_revenue
young adult	62143
middle-age	59197
adult	55978
senior	55763

5. Dashboard in Power BI

Finally, we built an interactive dashboard in **Power BI** to present insights visually.



6. Business Recommendations

- **Boost Subscriptions** – Promote exclusive benefits for subscribers.
- **Customer Loyalty Programs** – Reward repeat buyers to move them into the "Loyal" segment.
- **Review Discount Policy** – Balance sales boosts with margin control.
- **Product Positioning** – Highlight top-rated and best-selling products in campaigns.
- **Targeted Marketing –** Focus efforts on high-revenue age groups and express-shipping users.